

Discussion Unit: DSE Analytics

RDDs, Hello World, Data
Load

- Complete the big data classic; Hello World, (Word Count)
- Use DSE Analytics RDDs to load data from file
- Introduce RDD Transforms
- Introduce RDD Actions
- Write data to DSE

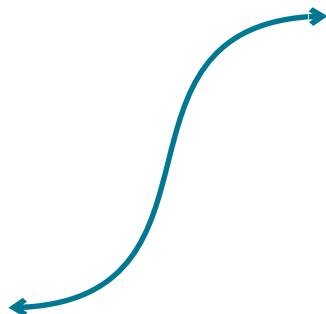


Discussion Lab:

Matching pairs – Match the attributes on the right with the areas on the left

DSE Analytics; RDDs, DataFrames, Datasets

DSE Analytics: RDD, DataFrame, Dataset



Are immutable



Reside (mostly) in memory



Are transparently distributed



Historically the core Spark abstraction



The fastest abstraction Spark uses



Make data across a cluster of machines look like a Scala collection



Occasionally manually redistribute



Rarely manually redistribute



Java, Scala only (not Python, R)



End of Discussion Lab:

Working Goal:

From: Maury_Atwater
To: DSE_HOTSHOT

Subject: Need this now !!!

Attached are 3 (count) SQL DDL files,
customer, orders, and items, and 3 (count)
ASCII Text CSV files.

I need these loaded into DSE now !!!

On customer.zipcode, derive an east coast,
west coast flag ("E-USA", "W-USA").
Also derive customer.company (company
name) to uppercase.

-MA



Maury Atwater, President
of Atwater's

111, Bob, Mary
222, Ted
333, Alice, Bob, Harold
444, Dave, Bob

Hello World ! (Word Count):



CSV file contents

```
val records = sc.textFile("file:///opt/stores_db/7545_HelloWorld.csv")  
val words  = records.flatMap( record => record.split(",").drop(1) )  
val counts1 = words.map( word => (word, 1) )  
val counts2 = counts1.reduceByKey{ case(x, y) => x + y }  
  
counts2.collect().foreach(println)
```

(Dave,1)
(Harold,1)
(Alice,1)
(Bob,3)
(Mary,1)
(Ted,1)



Output data



Unary transforms



Input parameters (made up)



Transformer symbol



Function body



Action

RDD: Class Types

(From previous page.)

`records.getClass()`

```
// res: Class[_ <: org.apache.spark.rdd.RDD[String]] =  
  class org.apache.spark.rdd.MapPartitionsRDD
```

`words.getClass()`

```
// res: Class[_ <: org.apache.spark.rdd.RDD[String]] =  
  class org.apache.spark.rdd.MapPartitionsRDD
```

`counts1.getClass()`

```
// res: Class[_ <: org.apache.spark.rdd.RDD[(String, Int)]] =  
  class org.apache.spark.rdd.MapPartitionsRDD
```

`counts2.getClass()`

```
// res: Class[_ <: org.apache.spark.rdd.RDD[(String, Int)]] =  
  class org.apache.spark.rdd.ShuffledRDD
```



Square brackets
mean What ?
(Best Scala answer: L)



Parenthesis mean
What ?
(Best answer: T)



Why

Lazy Evaluation

```
records.foreach(println)
// Above returns nothing, even after action upon it
//
// While records is of type RDD, right now it's basically a file pointer.
```

```
records.collect().foreach(println)
111, Bob, Mary
222, Ted
333, Alice, Bob, Harold
444, Dave, Bob
```



```
val records = sc.textFile("file:///opt/stores_db/7545_HelloWorld.csv")
val words   = records.flatMap( record => record.split(",").drop(1) )
val counts1 = words.map( word => (word, 1) )
val counts2 = counts1.reduceByKey{ case(x, y) => x + y }
```

Fluent API

// replaced by



Why an absolute
pathname ?

```
val records = sc.textFile("file:///opt/stores_db/7545_HelloWorld.csv")
val words   = records.flatMap( record => record.split(",").drop(1) )
val counts  = words.map( word => (word, 1) ).reduceByKey{ case(x, y) => x + y }
```

// replaced by

```
val records = sc.textFile("file:///opt/stores_db/7545_HelloWorld.csv")
val counts  = records.flatMap( record => record.split(",").drop(1) ).
    map( word => (word, 1) ).reduceByKey{ case(x, y) => x + y }
```

What does this do ?

```
val records = sc.textFile("file:///opt/stores_db/7545_HelloWorld.csv")  
val words   = records.flatMap( record => record.split(",").drop(1) )  
val counts  = words.map( word => (word, 1) ).reduceByKey{ case(x, y) => x + y }
```

// replaced by

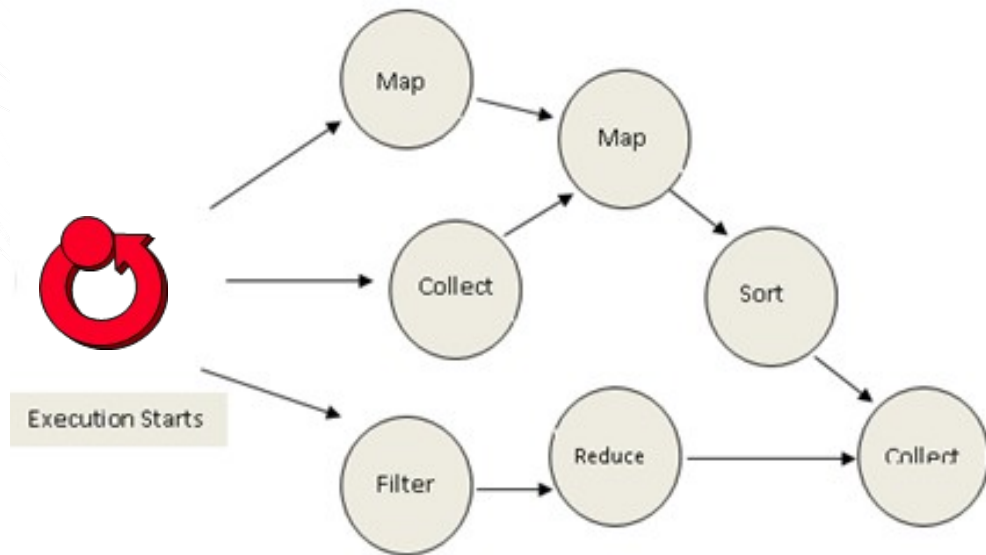
```
val records = sc.textFile("file:///opt/stores_db/7545_HelloWorld.csv")  
val words   = records.flatMap( record => record.split(",").drop(1) )  
val counts  = words.map( x => (x, 1) ).reduceByKey{ _ + _ }
```



Underscore means
What ?

DSE Analytics:

RDD Transforms and Actions



Source: <https://www.sigmoid.com/apache-spark-internals/>

How are RDDs Created :

- Parallelize a List
- Load data from stable storage
- Transform another RDD



```
case class My_Record (pk: Int, value: String)
```

```
val my_records = sc.parallelize(Seq(  
  new My_Record(555, "Luis, Earl"),  
  new My_Record(666, "Ann, Sally, Bob"),  
  new My_Record(111, "Bob, Tony")  
))
```

```
my_records.getClass()  
// res: Class[_ <: org.apache.spark.rdd.RDD[My_Record]] =  
  class org.apache.spark.rdd.ParallelCollectionRDD
```



Transform
or Action ?

Common RDD Unary Transforms (RDDs)

Transformation	Description
<code>filter(f)</code>	Return a new dataset formed by selecting those elements of the source on which function returns true.
<code>map(f)</code>	Return a new distributed dataset formed by passing each element of the source through a function.
<code>flatMap(f)</code>	Similar to map, but each input item can be mapped to 0 or more output items (so function should return a Sequence rather than a single item).
<code>distinct(n)</code>	Return a new dataset that contains the distinct elements of the source dataset.
<code>sample(...)</code>	Sample a fraction of the data, with or without replacement, using a given random number generator seed.



Common RDD Unary Transforms (RDDs)

Transformation	Description
<code>mapPartitions(f)</code>	Similar to map, but runs separately on each partition (block) of the RDD, so func must be of type <code>Iterator<T> => Iterator<U></code> when running on an RDD of type T.
<code>mapPartitionsWithIndex(f)</code>	Similar to mapPartitions, but also provides func with an integer value representing the index of the partition, so func must be of type <code>(Int, Iterator<T>) => Iterator<U></code> when running on an RDD of type T.
<code>pipe(envVars)</code>	Pipe each partition of the RDD through a shell command, e.g. a Perl or bash script. RDD elements are written to the process' stdin and lines output to its stdout are returned as an RDD of strings.
<code>coalesce(n)</code>	Decrease the number of partitions in the RDD to numPartitions. Useful for running operations more efficiently after filtering down a large dataset.
<code>repartition(n)</code>	Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network.

Common RDD Unary Transforms (RDDs)

Transformation	Description
<code>repartitionAndSort withinPartitions(p)</code>	Repartition the RDD according to the given partitioner and, within each resulting partition, sort records by their keys. This is more efficient than calling repartition and then sorting within each partition because it can push the sorting down into the shuffle machinery.

Common RDD Unary Transforms (Pair RDDs)

Transformation	Description
<code>groupByKey(n)</code>	When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs. ✓ te: If you are grouping in order to perform an aggregation (such as a sum or average) over each key, using <code>reduceByKey</code> or <code>aggregateByKey</code> will yield much better performance. ✓ te: By default, the level of parallelism in the output depends on the number of partitions of the parent RDD. You can pass an optional <code>numTasks</code> argument to set a different number of tasks.
<code>reduceByKey(f, n)</code>	When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function <code>func</code> , which must be of type <code>(V,V) => V</code> . Like in <code>groupByKey</code> , the number of reduce tasks is configurable through an optional second argument.
<code>aggregateByKey(...)</code>	When called on a dataset of (K, V) pairs, returns a dataset of (K, U) pairs where the values for each key are aggregated using the given combine functions and a neutral "zero" value. Allows an aggregated value type that is different than the input value type, while avoiding unnecessary allocations. Like in <code>groupByKey</code> , the number of reduce tasks is configurable through an optional second argument.
<code>sortByKey(asc, n)</code>	When called on a dataset of (K, V) pairs where K implements <code>Ordered</code> , returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean ascending argument.



Common RDD Binary Transforms (RDDs)

Transformation	Description
<code>union(otherRDD)</code>	Return a new dataset that contains the union of the elements in the source dataset and the argument.
<code>intersection(otherRDD)</code>	Return a new RDD that contains the intersection of elements in the source dataset and the argument.
<code>subtract(otherRDD)</code>	A new RDD Contains those elements from the source RDD that are not in otherRDD. Duplicates are allowed. Input RDDs must be union compatible.
<code>cartesian(otherRDD)</code>	When called on datasets of types T and U, returns a dataset of (T, U) pairs (all pairs of elements).



"union compatible" ?

Common RDD Binary Transforms (RDDs)

Transformation	Description
<code>join(otherRDD, n)</code>	When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (V, W)) pairs with all pairs of elements for each key. Outer joins are supported through <code>leftOuterJoin</code> , <code>rightOuterJoin</code> , and <code>fullOuterJoin</code> .
<code>cogroup(otherRDD, n)</code>	When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (Iterable<V>, Iterable<W>)) tuples. This operation is also called <code>groupWith</code> .

Common RDD Actions (RDDs)

Transformation	Description
<code>reduce(f)</code>	Aggregate the elements of the dataset using a function func (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
<code>collect()</code>	Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
<code>count()</code>	Return the number of elements in the dataset.
<code>first()</code>	Return the first element of the dataset (similar to take(1)).
<code>take(n)</code>	Return an array with the first n elements of the dataset.



Common RDD Actions (RDDs)

Transformation	Description
<code>takeSample(...)</code>	Return an array with a random sample of num elements of the dataset, with or without replacement, optionally pre-specifying a random number generator seed.
<code>takeOrdered(...)</code>	Return the first n elements of the RDD using either their natural order or a custom comparator.
<code>foreach(n)</code>	Run a function func on each element of the dataset. This is usually done for side effects such as updating an Accumulator or interacting with external storage systems. Note: modifying variables other than Accumulators outside of the foreach() may result in undefined behavior. See Understanding closures for more details.
<code>saveAsTextFile(p)</code>	Write the elements of the dataset as a text file (or set of text files) in a given directory in the local filesystem, HDFS or any other Hadoop-supported file system. Spark will call toString on each element to convert it to a line of text in the file.



Common RDD Actions (RDDs)

Transformation	Description
<code>saveAsSequenceFile(p)</code>	Write the elements of the dataset as a Hadoop SequenceFile in a given path in the local filesystem, HDFS or any other Hadoop-supported file system. This is available on RDDs of key-value pairs that implement Hadoop's Writable interface. In Scala, it is also available on types that are implicitly convertible to Writable (Spark includes conversions for basic types like Int, Double, String, etc).
<code>saveAsObjectFile(p)</code>	Write the elements of the dataset in a simple format using Java serialization, which can then be loaded using <code>SparkContext.objectFile()</code> .
<code>saveToCassandra(k, t, c)</code>	Stores all elements of the source RDD into a DSE table (t) in a given keyspace (k). Table columns may be specified if needed.



Common RDD Actions (Pair RDDs)

Transformation	Description
<code>countByKey()</code>	Only available on RDDs of type (K, V). Returns a hashmap of (K, Int) pairs with the count of each key.

About Actions: Optimization

Triggers work to start

- Calling an action says it is now time to evaluate
- Sent to the DAG Scheduler and off it goes
- Scalar value is returned to the driver
 - Avoid calling actions then transformations then actions then transformations as this means data is sent to driver, back out to the cluster, then to driver, back out, etc.
 - .. Lot of avoidable data movement
 - .. Can drastically slow down your performance
 - .. Can create an OOM on driver
 - I.e. load data from cassandra, run a map, collect, then flatmap, foreach, and then filter. If you would/could have done your filter, map, flatmap then collect would have saved a lot of data transmission, on a set of data you did not need
- Instead do all transformations up front
 - If you have multiple actions then with final RDD maybe apply a persistence (caching) mechanism on it to save, then do multiple actions (else you have to trigger the evaluation again)

Function examples from DSA-

Toy Story, 1995

```
rdd.filter(m => m.substring(m.length - 4, m.length).toInt == 2010)  
rdd.filter(m => m.split(",").last.trim.toInt == 2010)
```

Toy Story, 1995

```
rdd.map(m => m.substring(0, m.length - 6).length).  
  reduce{ case (x, y) => x + y }
```

Toy Story, 1995

```
rdd.map(m => m.substring(0, m.length - 6).length).  
  foreach{ 1 => totalCount += 1; totalLength += 1 }
```

```
println(totalLength.value / totalCount.value)  
println(totalLength.value.toDouble / totalCount.value)
```



What do each of these do ?

Function examples from DSA-

```
// String, Set(String)  
rdd.flatMap{case (m, g) => g}.distinct
```

```
// String, Int  
rdd.filter{ case (t, y) => y > 2010}
```

```
// data not provided  
rdd.map( m => (m, Set("aaa", "bbb")))
```

```
// rdd1 = org.apache.spark.rdd.RDD[String]  
rdd2 = rdd1.cartesian(rdd1).filter{case (s, t) => s != t}
```

```
// hint: outputs, RDD[(String, String)]
```



What do each of these do ?

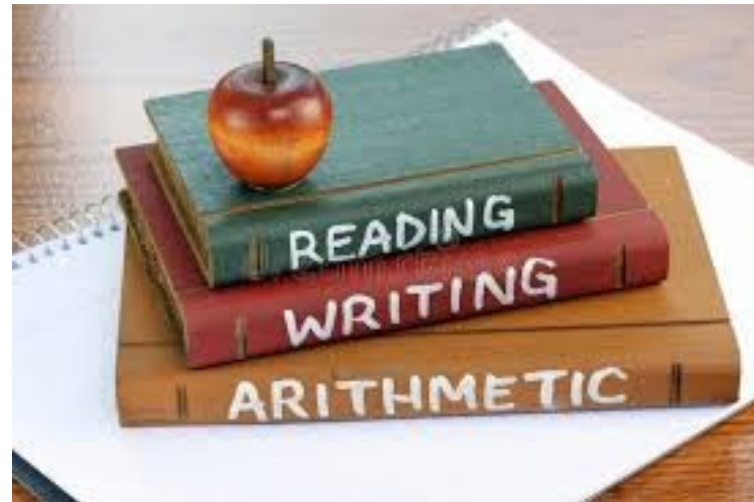


Why do we sometimes pass {} to a transform, and other times () ?

Best answer: E and F

DSE Analytics:

Reading and writing to DSE
using RDDs



```
DROP KEYSPACE IF EXISTS ks_7545;  
CREATE KEYSPACE ks_7545 WITH REPLICATION =  
    {'class': 'SimpleStrategy',  
     'replication_factor': 1};
```

```
USE ks_7545;  
CREATE TABLE hello_world  
(  
    pk          INT PRIMARY KEY,  
    value       TEXT  
);
```

```
INSERT INTO hello_world (pk, value)  
VALUES (111, 'Bob, Mary'      );
```

```
    ...  
VALUES (222, 'Ted'            );  
VALUES (333, 'Alice, Bob, Harold');  
VALUES (444, 'Dave, Bob'      );
```

CQL Assets to work with

**OLD
OLD
OLD**



Reading from DSE using RDDs

```
val rows = sc.cassandraTable("ks_7545", "hello_world")  
rows.collect().foreach(println)
```

```
CassandraRow{pk: 111, value: Bob, Mary}  
CassandraRow{pk: 222, value: Ted}  
CassandraRow{pk: 444, value: Dave, Bob}  
CassandraRow{pk: 333, value: Alice, Bob, Harold}
```

```
rows.getClass()  
// res: Class[_ <: com.datastax.spark.connector.rdd.CassandraTableScanRDD[  
  com.datastax.spark.connector.CassandraRow]] =  
  class com.datastax.spark.connector.rdd.CassandraTableScanRDD
```



```
case class My_Record (pk: Int, value: String)
```

```
val my_records = sc.parallelize(Seq(  
  new My_Record(555, "Luis, Earl"),  
  new My_Record(666, "Ann, Sally, Bob"),  
  new My_Record(111, "Bob, Tony")  
))
```

```
my_records.getClass()  
// res: Class[_ <: org.apache.spark.rdd.RDD[My_Record]] =  
  class org.apache.spark.rdd.ParallelCollectionRDD
```

```
// SomeColumns is a keyword
```

```
//
```

```
my_records.saveToCassandra("ks_7545", "hello_world",  
  SomeColumns("pk", "value"))
```

Writing to DSE using RDDs

Case class; like struct
in C, POJO in Java

SomeColumns is a
keyword



Insert verify

```
val rows = sc.cassandraTable("ks_7545", "hello_world")  
rows.collect().foreach(println)
```

CassandraRow{pk: 111, value: Bob, Tony}
CassandraRow{pk: 222, value: Ted}
CassandraRow{pk: 444, value: Dave, Bob}
CassandraRow{pk: 555, value: Luis, Earl}
CassandraRow{pk: 666, value: Ann, Sally, Bob}
CassandraRow{pk: 333, value: Alice, Bob, Harold}



there's
something
about
mary



Source: https://www.rottentomatoes.com/m/theres_something_about_mary/

Is a PK retrieval still a (Table Scan) ?

```
val rows = sc.cassandraTable[My_Record]("ks_7545", "hello_world").  
  where("pk=111").  
  select("pk", "value").  
  as( (i:Int, s:String ) => new My_Record(i, s) )  
rows.collect().foreach(println)  
  //  
rows.getClass()  
// res: Class[_ <: com.datastax.spark.connector.rdd.CassandraTableScanRDD[My_Record]] =  
  class com.datastax.spark.connector.rdd.CassandraTableScanRDD
```



Pick one, don't need both

Various casts, on read

```
val rows = sc.cassandraTable("ks_7545", "hello_world")  
// com.datastax.spark.connector.rdd.CassandraTableScanRDD[  
  com.datastax.spark.connector.CassandraRow]]
```

```
val rows = sc.cassandraTable[ (Int, String) ]("ks_7545", "hello_world")  
// rdd.CassandraTableScanRDD[(Int, String)]
```

```
case class My_Record (pk: Int, value: String)  
//
```

```
val rows = sc.cassandraTable[My_Record]("ks_7545", "hello_world")  
//rdd.CassandraTableScanRDD[My_Record]]
```

```
val rows = sc.cassandraTable[My_Record]("ks_7545", "hello_world").  
  select("pk", "value").  
  as( (i:Int, s:String ) => new My_Record(i, s) )  
// rdd.CassandraTableScanRDD[My_Record]]
```



Scala REPL
versus Scala
compiler


```
CREATE TABLE customer
(
  customer_num
  fname
  lname
  company
  address1
  address2
  city
  state
  zipcode
  phone
);
```

```
INT PRIMARY KEY,
TEXT,
TEXT,
TEXT,
TEXT,
TEXT,
TEXT,
TEXT,
TEXT
```

Working
with CSVs,
RDDs,
writing

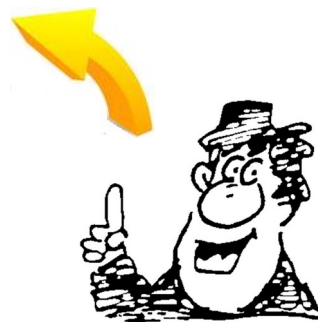
Working with CSVs, RDDs, writing

```
val rows20 = sc.textFile(
  "file:///opt/stores_db/customer.csv")

val rows21 = rows20.map ( line => line.split (","))

val rows22 = rows21.map ( p => My_Record ( p(0).toInt,
  p(1).toString, p(2).toString, p(3).toString, p(4).toString,
  p(5).toString, p(6).toString, p(7).toString, p(8).toString,
  p(9).toString ))

rows22.saveToCassandra("ks_7545", "customer",
  SomeColumns( "customer_num"      , "fname",
    "lname"          , "company"      ,
    "address1"       , "address2"    ,
    "city"           , "state"       ,
    "zipcode"        , "phone"       ))
```



Not our first choice:
positional, fragile

Working with CSVs, RDDs, writing

```
val My_Schema = StructType(Array(  
  StructField("customer_num" , IntType, true),  
  StructField("fname"        , StringType, true),  
  StructField("lname"        , StringType, true),  
  StructField("company"      , StringType, true),  
  StructField("address1"     , StringType, true),  
  StructField("address2"     , StringType, true),  
  StructField("city"         , StringType, true),  
  StructField("state"        , StringType, true),  
  StructField("zipcode"      , StringType, true),  
  StructField("phone"        , StringType, true)  
))
```



Technically, this CSV read is into a DataFrame; hence, the **cast to RDD**

```
val rows30 =  
    spark.read.schema(My_Schema).csv(  
        "file:///opt/stores_db/customer.csv")
```

```
val rows31 = rows30.rdd
```

```
rows31.saveToCassandra("ks_7545", "customer",  
    SomeColumns( "customer_num"      , "fname",  
        "lname"                      , "company",  
        "address1"                   , "address2",  
        "city"                       , "state",  
        "zipcode"                    , "phone"      ))
```

Working
with CSVs,
RDDs,
writing

Back to Atwater's



Working Goal:

From: Maury_Atwater
To: DSE_HOTSHOT

Subject: Need this now !!!

Attached are 3 (count) SQL DDL files,
customer, orders, and items, and 3 (count)
ASCII Text CSV files.

I need these loaded into DSE now !!!

On customer.zipcode, derive an east coast,
west coast flag ("E-USA", "W-USA").
Also derive customer.company (company
name) to uppercase.

-MA



Maury Atwater, President
of Atwater's

```
CREATE TABLE customer_plus_derived
```

```
(  
  customer_num          INT PRIMARY KEY,  
  fname                 TEXT,  
  lname                 TEXT,  
  company               TEXT,  
  address1              TEXT,  
  address2              TEXT,  
  city                  TEXT,  
  state                 TEXT,  
  zipcode               TEXT,  
  phone                 TEXT,  
  company_upshift       TEXT,  
  ew_flag               TEXT  
);
```

```
INSERT INTO customer
```

```
(customer_num, fname, lname, company, address1,  
  address2, city, state, zipcode, phone)  
VALUES ( 101, 'Ludwig', 'Pauli', 'All Sports Supplies',  
  '213 Erstwild Court', '', 'Sunnyvale', 'CA', '94086',  
  '408-789-8075');
```

SQL Assets you
work with in the
lab to follow

111, Bob, Mary
222, Ted
333, Alice, Bob, Harold
444, Dave, Bob

From Before ..
(or, specifically the CSV handling
code shown)

```
val records = sc.textFile("file:///opt/stores_db/7545_HelloWorld.csv")  
val words  = records.flatMap( record => record.split(",").drop(1) )  
val counts1 = words.map( word => (word, 1) )  
val counts2 = counts1.reduceByKey{ case(x, y) => x + y }  
  
counts2.collect().foreach(println)
```

(Dave,1)
(Harold,1)
(Alice,1)
(Bob,3)
(Mary,1)
(Ted,1)



Add the first column derivation

```
val recs_withDerived = counts2.map { case(name, cnt) =>  
  (name, cnt, name.toUpperCase) }
```

```
recs_withDerived.collect().foreach(println)
```

```
( Dave,1, DAVE)  
( Harold,1, HAROLD)  
( Alice,1, ALICE)  
( Bob,3, BOB)  
( Mary,1, MARY)  
( Ted,1, TED)
```

CamelCase
lowercase
MiXeDcAsE
Proper Case
Title Case
UPPERCASE

```
object MyFunctions {  
  def generateFlag(arg: String) : String = {  
    if (arg > "M")  
      "E-USA"  
    else  
      "W-USA"  
  }  
}  
  
val recs_withDerived = counts2.map { case(name, cnt) =>  
  (name, cnt, name.toUpperCase,  
    MyFunctions.generateFlag(name) ) }  
recs_withDerived.collect().foreach(println)  
  
( Dave,1, DAVE,W-USA)  
( Harold,1, HAROLD,W-USA)  
( Alice,1, ALICE,W-USA)  
( Bob,3, BOB,W-USA)  
( Mary,1, MARY,W-USA)  
( Ted,1, TED,W-USA)
```

Add the second column derivation



You will
use
zipcode

Extra
credit for
an integer

Using Records

```
case class My_Record (pk: Int, value: String)
//
val rows = sc.cassandraTable[My_Record]("ks_7545", "hello_world")
rows.collect().foreach(println)
```

```
My_Record(111,Bob, Tony)
My_Record(222,Ted)
My_Record(444,Dave, Bob)
My_Record(555,Luis, Earl)
My_Record(666,Ann, Sally, Bob)
My_Record(333,Alice, Bob, Harold)
```

Using Records

```
val rows2 = rows.map { case(row) => (row.value, row.pk,  
    row.value.toUpperCase, MyFunctions.generateFlag(row.value) ) }
```

```
rows2.collect().foreach(println)
```

```
(Bob, Tony,111,BOB, TONY,W-USA)
```

```
(Ted,222,TED,E-USA)
```

```
(Dave, Bob,444,DAVE, BOB,W-USA)
```

```
(Luis, Earl,555,LUIS, EARL,W-USA)
```

```
(Ann, Sally, Bob,666,ANN, SALLY, BOB,W-USA)
```

```
(Alice, Bob, Harold,333,ALICE, BOB, HAROLD,W-USA)
```



End of Unit:



Additional Detail:

Other UIs: Using DSE Analytics w/ Notebooks



DSE Studio is a notebook (based on Zeppelin); there are three main notebooks in addition to Studio

- Jupyter (<http://jupyter.org/>)
- Zeppelin (<https://zeppelin.apache.org/>)
- Spark Notebook (<http://spark-notebook.io/>)

DSE 6.0+ there are options to run notebooks against DSE more natively

- `dse exec`
- Launches a shell command with environment variables set by DSE. Example, `dse exec bin/spark-notebook`

More Scala: Munging records ..

You've loaded data before; it's all about the munge-

```
case class Person (fname: String, lname: String, age: Int)
```

```
// Emily gets married
```

```
val emily1 = Person("Emily", "Means", 40)
```

```
val emily2 = emily1.copy(lname = "Walls", age = 42)
```

```
// also
```

```
// val emily1 = Person(lname = "Means", fname = "Emily", age = 40)
```

```
val fullName = (p: Person) => s"${p.fname} ${p.lname}"
```

```
val xx = fullName(emily1)
```

```
// xx: String = Emily Means
```


More Scala: Functions as data

```
val double = (i: Int) => i * 2  
val isEven = (i: Int) => i % 2 == 0
```

```
val x = 42  
val y = double(x)  
val y = isEven(x)
```

```
val ints = List(1, 2, 4)
```

```
// filter expects a boolean  
// map expects a list  
//  
val y = ints.map(double)  
val y = ints.filter(isEven)
```



More Scala: Functions as data

```
val isEven = (i: Int) => i % 2 == 0
```

```
// Also,
```

```
val isEven = (i: Int) => { i % 2 == 0 }
```

```
val isEven = (i: Int) => if ( i % 2 == 0 ) true else false
```

```
val isEven = (i: Int) => { if ( i % 2 == 0 ) true else false }
```

```
val isEven = (i: Int) => {  
  if ( i % 2 == 0 ) {  
    true  
  } else {  
    false  
  }  
}
```

isEven

(i: Int)

=>

i % 2 == 0

field name

input params

transformer symbol

function body

```
case class My_Record (pk: Int, value: String)
```

```
val my_records = sc.parallelize(Seq(  
  new My_Record(555, "Luis, Earl"),  
  new My_Record(666, "Ann, Sally, Bob"),  
  new My_Record(111, "Bob, Tony"),  
  new My_Record(111, "Jennifer")  
))
```

```
my_records.count
```

```
val my_records_asDF = my_records.toDF
```

```
my_records_asDF.count
```

```
...
```

More Scala:
Helper function,
toDF

...

```
my_records_asDF.show
my_records_asDF.orderBy("value").show
```

```
+---+-----+
| pk|          value|
+---+-----+
|666|Ann, Sally, Bob|
|111|          Bob, Tony|
|555|          Luis, Earl|
+---+-----+
```

```
my_records_asDF.orderBy("value").take(1)
my_records_asDF.printSchema()
root
 |-- pk: integer (nullable = false)
 |-- value: string (nullable = true)
```

More Scala:
Helper function,
toDF

More Scala: Helper function, toDF

```
my_records_asDF.groupBy("pk").count.show()
my_records_asDF.groupBy("pk").count.orderBy("pk").show
my_records_asDF.groupBy("pk").count.orderBy("count").show
my_records_asDF.groupBy("pk").count.orderBy($"count".desc).show()
my_records_asDF.groupBy("pk").count.orderBy("pk").show
```

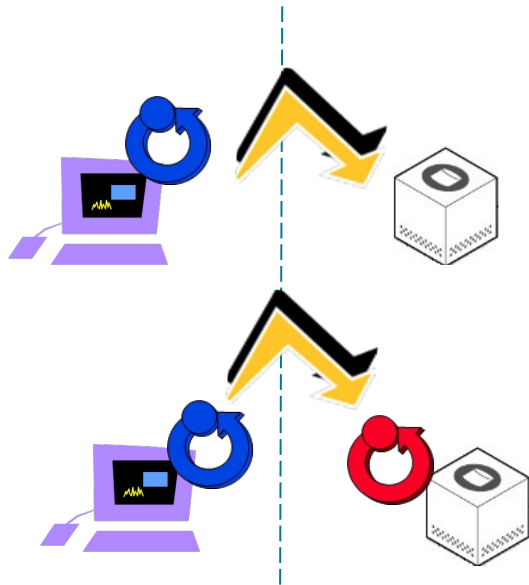
There are JDBC, ODBC, Parquet, (other) connectors built into Spark, but there is not a (Cassandra) one

DataStax provides 2 drivers that work interactively

- DSE Java Driver (See Discussion Unit, 6244/6245) (not Spark)
- Spark Cassandra Connector (Spark)
 - Open source, AND enterprise license features
 - Spark objects, activities

These allow you to connect to DSE directly from Spark, like using a JDBC driver, more

[What] is interacting with DSE ?



DSE Java Driver-

On top of the open source driver sits the DSE Driver-

- Specific extensions for DSE
 - Authenticator implementations that use the authentication scheme negotiation in the server-side DseAuthenticator
 - Value classes for geospatial types, and type codecs that integrate them seamlessly
 - Continuous Paging
 - DSE graph integration
 - <https://github.com/datastax/java-dse-driver>
- Only allowed to use if you have a valid DSE License

DSE Spark (Cassandra) Connector-

- Exposes (Cassandra) tables as Spark RDDs, DataFrames, Datasets
- Read, write, and more
- Execute arbitrary CQL queries in your Spark applications
- Used with the Cassandra Java Driver
- Open Source Apache 2.0 license
- DataStax Github Repository, <https://github.com/datastax/spark-cassandra-connector>

Use cases:

- Streaming data into DSE
- Analyzing the data in place
- Migrating from bad data model

Spark Client Web UI: Extra TAB when streaming

